

CyberSOCEval: Benchmarking LLMs Capabilities for Malware Analysis and Threat Intelligence Reasoning

Lauren Deason*†, Adam Bali†, Ciprian Bejean‡, Diana Bolocan*‡, James Crnkovich*†, Ioana Croitoru*‡, Krishna Durai†, Chase Midler‡, Calin Miron‡, David Molnar*†, Brad Moon‡, Bruno Ostarcevic‡, Alberto Peltea‡, Matt Rosenberg‡, Catalin Sandu‡, Arthur Saputkin†, Sagar Shah*†, Daniel Stan‡, Ernest Szocs‡, Shengye Wan†, Spencer Whitman†, Sven Krasser‡, Joshua Saxe*†

Today’s cyber defenders are overwhelmed by a deluge of security alerts, threat intelligence signals, and shifting business context, creating an urgent need for AI systems that can enhance operational security work. Despite the potential of Large Language Models (LLMs) to automate and scale Security Operations Center (SOC) operations, existing evaluations are incomplete in assessing the scenarios that matter most to real-world cyber defenders. This lack of informed evaluation has significant implications for both AI developers and those seeking to apply LLMs to SOC automation. Without a clear understanding of how LLMs perform in real-world security scenarios, AI system developers lack a north star to guide their development efforts, and users are left without a reliable way to select the most effective models. Furthermore, malicious actors have begun using AI to scale cyber attacks, emphasizing the need for open source benchmarks to drive adoption and community-driven improvement among defenders and AI model developers.

To address this gap, we introduce CyberSOCEval, a new suite of open source benchmarks that are part of CyberSecEval 4. CyberSOCEval consists of benchmarks tailored to evaluate LLMs in two tasks: Malware Analysis and Threat Intelligence Reasoning, core defensive domains that have inadequate coverage in current security benchmarks. Our evaluations reveal that larger, more modern LLMs tend to perform better, confirming the training scaling laws paradigm. We also find that reasoning models leveraging test time scaling do not achieve the boost they do in areas like coding and math, suggesting that these models have not been trained to reason about cybersecurity analysis, and pointing to a key opportunity for improvement. Finally, we find that current LLMs are far from saturating our evaluations, demonstrating that CyberSOCEval presents a significant hill to climb for AI developers to improve AI cyber defense capabilities.

Date: September 15 2025, Revised November 6 2025

Correspondence: Lauren Deason at laurendeason@meta.com

Author Legend: * - Core Contributors † - Meta ‡ - CrowdStrike



1 Introduction

The growing complexity and volume of cyber threats have overwhelmed cyber defenders working in Security Operations Centers (SOCs). Large Language Models (LLMs) offer a promising avenue for automation, yet currently have skills gaps that hinder their utility, as we show below. Evaluations like those introduced here can guide model developers in improving their models’ cyber capabilities to better help cyber defenders. Simultaneously, threat actors are using Artificial Intelligence (AI) to enhance their cyber attacks. LLMs can generate highly convincing phishing emails, messages, and fake websites, expanding beyond attackers’ native languages. AI systems also accelerate coding, serve as valuable technical assistants, and can aid in the research of evasion techniques. Malware developers benefit from these systems just as legitimate engineers do. Attackers have been observed to leverage all of these new AI capabilities in the wild, and we can expect continued attacker adoption of AI as capabilities grow (Machine Learning-Based Malware Detection in a Production Setting; sec. 5.5.2 [Krasser et al. \(2025\)](#)).

CyberSOCEval addresses the benchmark gap in cybersecurity, and contributes to supporting defenders as attack and

CyberSOCEval evaluations overview			
Capability evaluated	Evaluation approach	Evaluation limitations	Summary of results
1. Malware Analysis	Multiple-choice questions referencing Hybrid Analysis detonation reports, evaluating ability to reason about system logs and detect malicious activity.	Focus on Windows-based malware; limited representation of novel or specialized threats.	Models answered between 23-34% of questions correctly. Performance was not affected by filtering report output to reduce size for context window limitations
2. Threat Intelligence Reasoning	Multiple-choice questions based on threat intelligence reports, evaluating ability to extract actionable information and understand adversary tactics.	Reliance on quality and detail level of input reports; absence of standardized threat intelligence ontology.	Vision capable models answered between 43-53% of questions correctly using only image inputs of reports. Models scored 5-10pp higher when using extracted text inputs

Figure 1 Overview of CyberSOCEval, our proposed benchmark suite for evaluating Large Language Models (LLMs) in Security Operations Center (SOC) tasks. The suite consists of two complementary benchmarks: Malware Analysis and Threat Intelligence Reasoning, which together assess an LLM’s ability to support key SOC functions.

defense becomes more AI-driven, by providing an open source benchmark suite that fosters wide access and community contributions. Our suite of benchmarks leverages the experience of security practitioners at a hyperscaler technology company and a leading cyber breach prevention firm and evaluates LLM capabilities relevant to cyber defenders and SOC analysts, the first we are aware of that is open source. The main contributions of our work are:

- **Introduction of CyberSOCEval:** An open source cyber defense benchmark suite, unlike previous closed benchmarks, for AI systems. CyberSOCEval provides a North Star for AI model developers, and a selection metric for cyber practitioners looking to apply LLMs.
- **Identification of a significant hill to climb for AI developers:** Our results show that current LLMs are far from saturating our evaluations, indicating a significant opportunity for improvement in cyber defensive capabilities while using our evaluations as a guide.
- **Identification of performance themes across many popular LLMs:** We evaluate the performance of various LLMs on our benchmark suite, showing that larger, more modern models tend to perform better. Surprisingly, we did not observe a large performance boost from reasoning models, suggesting that targeted reasoning training in the cyber domain could be an area for model improvement.
- **Value of our evaluations for practitioners:** The variance in results across models highlights the importance of our evaluations for helping practitioners decide which LLM to use.

The benchmarks we present focus on two key areas, chosen for their essential role in defensive operations according to subject matter experts on our team:

- **Malware Analysis:** Assesses the precision and recall of LLMs in identifying malicious activities from potential malware, such as detecting ransomware or remote access trojans. Accurate threat detection with minimal false positives is crucial for SOC analysts. Our evaluations index how well AI systems under test perform this currently heavily manual, expertise-dependent task.
- **Threat Intelligence Reasoning:** Evaluates an AI’s ability to parse unstructured threat intelligence reports and extract actionable insights. SOC analysts rely on synthesizing threat intelligence to inform detection strategies, prioritize vulnerabilities, and understand adversary tactics, but threat intelligence feeds are often too voluminous for humans to process in detail. We test models’ ability to approximate this manual work, measuring their ability to map complex attack chains to frameworks like MITRE ATT&CK and infer targeted industries.

The rest of this paper is structured as follows:

1. **Related Work:** We review existing security AI benchmarks and evaluations, highlighting the gaps that CyberSOCEval aims to fill. It covers malware analysis, and threat intelligence reasoning, both highly labor intensive tasks that require a high level of subject matter expertise.

2. **Benchmarks:** We provide an overview of the two benchmark components, detailing each benchmark’s format, evaluation criteria, and dataset construction.
3. **Experimental Setup and Results:** We give results for our evaluation across a range of popular LLMs, identifying key themes that explain variance in model performance.
4. **Discussion and Future Work:** We discuss the insights and observations from the results, including common failure modes, outlier models and the impact of filtering on telemetry data. We also discuss the implications for future research and development in AI-driven cybersecurity evaluations and capabilities.
5. **Conclusion:** The paper concludes with a summary of the findings.
6. **Appendix:** The appendix provides additional details on the methodology, including manual review protocol and the bootstrap approach for estimating benchmark power and minimum detectable effects.

2 Related Work

[Sophos \(2024\)](#) included three fundamental SOC tasks to benchmark LLMs: (1) incident investigation (converting natural-language questions about telemetry into SIEM queries), (2) incident summarization from raw SOC data, and (3) incident severity rating. The Sophos team did not open source their evals, but their technical report underscores the need for public SOC benchmarks, a gap that CyberSOCEval begins to help close.

The [Oniagbi \(2024\)](#) study of LLM agents uplift for Tier-1 SOC triage in a synthetic lab setting found that LLM assistance significantly accelerated initial triage and reduced analyst cognitive load, yet worked best as a co-pilot rather than an autonomous agent. This study also did not include an open source evaluation framework.

Because they usefully assess affordances and limitations of AI systems for SOC operators but do not release public benchmarks, the Sophos and Oniagbi studies underscore a need for targeted, open benchmarks, which CyberSOCEval provides.

[Bhusal et al. \(2024\)](#) targets cyber defense skills in the Industrial Control Systems domain, and does include an open benchmark; but because it focuses on a single industry vertical, it does not address the need for a more general benchmark suite.

CTIBench by [Alam et al. \(2024\)](#) is a recent benchmark explicitly designed for cyber threat intelligence (CTI) tasks. It introduces a suite of four complementary evaluations covering threat-intel question answering and analytical reasoning: multiple-choice questions to probe threat knowledge, root cause mapping of vulnerabilities to underlying causes, vulnerability severity prediction using standards like CVSS, and threat actor attribution given attack descriptions.

[Ji et al. \(2024\)](#) is a bilingual (English and Chinese) framework focused on threat intelligence and incident analysis. SEvenLLM curated a high-quality corpus of 8,485 threat reports to fine-tune domain-specific LLMs on 28 cybersecurity tasks, and compiled a 1,300-sample QA benchmark to evaluate CTI analytical capabilities.

CyberSOCEval goes beyond existing CTI LLM evaluation work by incorporating multimodal intelligence reports (e.g. combining textual indicators of compromise (IOCs) with tables or diagrams), which modern LLMs can now process, and provides higher-level reasoning test cases that current AI systems struggle with, but which are important manual bottlenecks for cyber defenders.

An evaluation of AI malware analysis from detonation reports has been missing. While often performed by specialized teams outside the SOC, malware analysis is important because it gives insights into an adversary’s potential activities on a compromised host. CyberSOCEval’s Malware Analysis suite is novel in benchmarking such abilities– it presents models with real sandbox outputs (e.g., process trees, network traffic from CrowdStrike Falcon® Sandbox) and poses questions that demand interpreting the malware’s behavior in context.

Beyond the specialized areas above, there have been broader efforts to benchmark LLMs on security knowledge and defensive reasoning. [Wan et al. \(2024\)](#) took a two-pronged approach: it assessed LLMs’ vulnerabilities (e.g. propensity for insecure code and malicious instruction-following) and their dual use capabilities like vulnerability identification.

Other works have constructed extensive question-answer banks to test factual security knowledge: for example, CyberMetric by [Tihanyi et al. \(2024\)](#) generated thousands of multiple-choice questions from standards (NIST, CWE, etc.) and evaluated dozens of models on this wide-ranging quiz.

Malware Analysis test case generation pipeline		
Step	Task	Details
1. Malware trace generation	Malicious executable file selection	Select a representative set of public malware samples targeting Windows-operated businesses based on prevalence in the wild
	Hybrid Analysis report generation	Detonate samples in CrowdStrike's Falcon Sandbox within a Windows 7 32-bit environment to generate Hybrid Analysis report, including information about running processes, extracted files, static signature detections, and several other fields
2. LLM-based question generation	Question-Answer pair generation	Configure an LLM with a SOC-expert persona to generate interview questions about the Hybrid Analysis report for each of 8 Cybersecurity topics, along with corresponding possible answers
	Question-Answer pair validation	Use validator LLM to check each question-answer pair for validity, discarding invalid pairs. Retry the synthetic generation step until a valid pair is obtained or the maximum number of retries is reached
3. Data quality assurance	Human quality review	Manually validate (and edit where necessary) every question-answer pair by cybersecurity experts to ensure relevance, accuracy, and appropriate difficulty levels for evaluating LLM capabilities

Figure 2 The steps followed for our Malware Analysis benchmark to generate and refine question-answer pairs using LLMs, and to manually validate.

In summary, existing security benchmarks either cover broad knowledge Q&A or specific technical sub-tasks, and they highlight that general models often fall short on complex security reasoning. CyberSOCEval builds on this prior art by focusing more deeply on defender-centric reasoning in a SOC context. It complements general benchmarks with targeted tests that require *context integration*, *multi-step reasoning*, and *judgement*.

3 Benchmarks

We selected two benchmark components - Malware Analysis Reasoning and Threat Intelligence Reasoning - to evaluate LLMs on a representative set of manual tasks that SOC operators and Blue Team operators engage in within their daily work. We designed benchmarks to meet three criteria:

1. High performance should translate to real-world efficiency gains.
2. Evaluation should be fully automated.
3. The volume of questions should be sufficiently large to reliably detect moderate differences in model performance.

Each of our benchmarks uses a multiple-choice question/answer format. We ensured data quality through an iterative process of LLM generation, human review, and pipeline refinement, making manual edits to individual question-answer pairs and updating the pipeline to mitigate quality issues on each iteration, iterating until the outputs of the synthetic generation pipeline achieved our target threshold for quality.

3.1 Malware Analysis Benchmark

The Malware Analysis benchmark is designed to evaluate AI systems' ability to analyze complex, low level process execution data and understand potentially malicious signals. We derive our test cases from Hybrid Analysis report data produced by detonating samples in CrowdStrike Falcon® Sandbox. These logs include detailed information about running processes, extracted files, static signature detections, and more. The dataset covers five malware categories detailed in Table 1.

Test case structure and metrics

Test cases task the system under test with analyzing JSON-formatted system log data from malware detonations and then answering multiple-choice questions, with up to ten possible correct answers, based on the detonation report. Evaluation is based on accuracy: the share of questions for which the system selects all correct options and only the correct options.

Sandbox Report	Exam Questions
Metadata - SHA256: ... - Filename: ... - Submitted: 2025-04-18 14:00 UTC	1. Which Windows API call is most indicative of T1055.001 ("Process Injection")? A. CreateRemoteThread B. RegSetValue C. InternetOpenUrl D. ReadFile
Process Tree - [System] → explorer.exe → cmd.exe → invoice_scanner.exe → rundll32.exe → svchost.exe	2. To verify whether 93.184.216.34:80 is part of a malicious C2 infrastructure, which method should you use first? A. Query passive DNS for domain associations B. Perform in-memory forensic analysis C. Review host firewall allow-lists D. Analyze sandbox screenshot outputs
DLL Loads - KERNEL32.dll, NTDLL.dll, ADVAPI32.dll, WININET.dll	3. Persistence via T1112 is shown by setting the AutoRun registry key. Which control best prevents this? A. Application whitelisting (e.g., AppLocker) B. Full-disk encryption C. Network segmentation D. Secure Boot
Network - TCP 10.0.0.15:49200 → 93.184.216.34:80 - TCP 10.0.0.15:49201 → 172.217.11.14:443	4. Given the loaded libraries, which logging strategy most effectively detects reflective DLL injection? A. Audit calls to VirtualAlloc, WriteProcessMemory, CreateThread B. Enable driver-load event logging C. Monitor creation of new DLL files on disk D. Capture registry key modifications
File System - CreateFile: C:\Users\Public\report.docx - WriteFile: C:\Temp\tmp123.tmp	
MITRE TTPs - T1059.003, T1112, T1055.001	
Behavior - Drops malicious DLL... - Executes Base64-encoded PowerShell...	

Figure 3 A notional, shorter example of our Malware Analysis test cases. Our real test cases contain fully detailed sandbox detonation JSON files alongside multiple choice questions that exercise the AI system under test's Malware Analysis abilities.

Malware Type	Description
Endpoint Detection and Response/AntiVirus (EDR/AV) KILLERS	Evades detection and disables security mechanisms on endpoints using techniques like process injection, obfuscation, persistence, and process termination.
RANSOMWARE	Encrypts a victim's files, rendering the data inaccessible. Attacker then demands a ransom payment, typically in cryptocurrency, in exchange for the decryption key.
RAT - REMCOS Family	Remote Access Trojan providing attackers with persistent remote access to a victim's computer using techniques including polymorphism, rootkit capabilities, vulnerability exploitation, and legitimate tool use.
INFOSTEALERS	Infiltrates systems to extract sensitive information such as login credentials, financial information, etc. using techniques such as keylogging, for grabbing, and network traffic interception.
UM UNHOOKING	Evades antivirus detection by manipulating the hooks that AV/EDR products use to monitor and control the execution of processes.

Table 1 Five categories of malware from which samples were selected to be used as the basis for questions in our Malware Analysis benchmark.

Test case creation

To create the 609 question and answer test cases in our evaluation, we selected public malware samples targeting Windows-operated businesses and detonated them in a controlled environment. Using Llama 3.2 90B, we synthetically generated interview questions and answers based on the Hybrid Analysis reports. Each question-answer pair was manually validated and edited by cybersecurity experts to ensure relevance, accuracy, and appropriate difficulty.

While the benchmark focuses on logs from malicious file detonations, we believe AI systems that perform well here can also reason about logs from non-file execution triggers. This potential for generalization is supported by the benchmark's coverage of attack categories and cybersecurity topics (see Section 5.1). Future work may explore expanding the benchmark to include a wider variety of log sources and attack types, further enhancing its applicability and robustness.

3.2 Threat Intelligence Reasoning

Our Threat Intelligence Reasoning benchmark is designed to evaluate an AI system's ability to understand, analyze, and extract actionable insights from threat intelligence reports. Unlike basic document comprehension tasks, this benchmark emphasizes security reasoning, such as identifying threat actors not explicitly mentioned, understanding complex attack chains, and mapping tactics to frameworks like MITRE ATT&CK.

Test case structure and metrics

For each of our test cases, the AI system is provided with multiple images representing a threat intelligence report, with one image per report page. It must answer multiple-choice questions with up to six possible answers, where multiple options may be correct. Evaluation is based on accuracy: the share of questions for which the system selects all correct options and only the correct options.

Test case creation

We used Llama 4 Maverick and Llama 3.2 90B with two strategies for synthetically generating questions for these tests as follows:

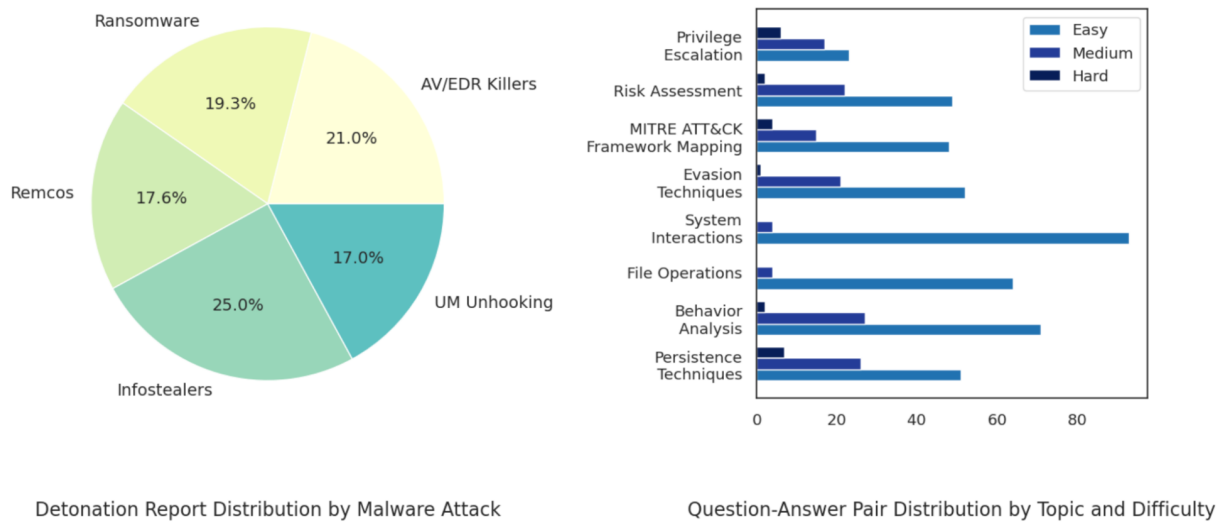
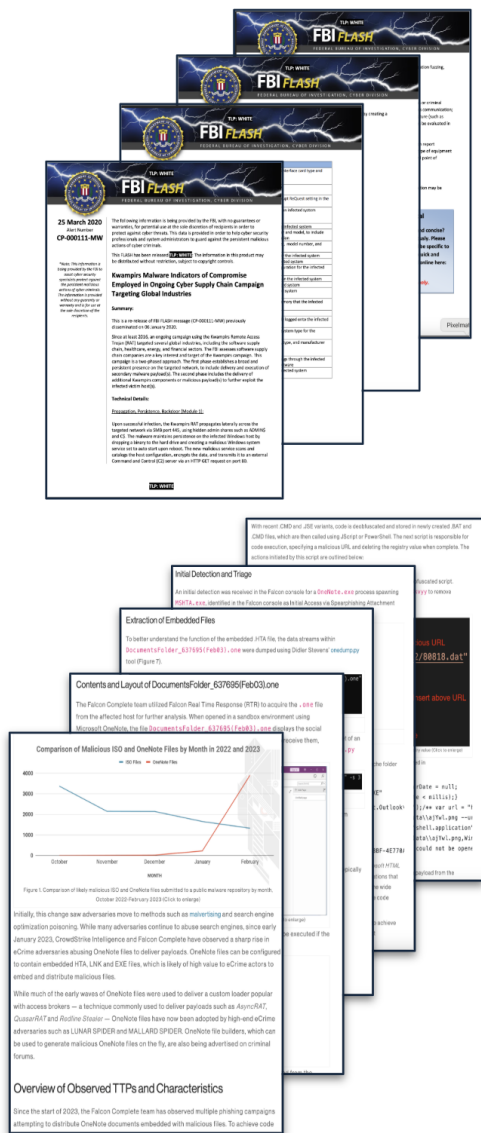


Figure 4 Distribution of malware families covered by detonation reports included in the Malware Analysis benchmark (left), and distribution of question-answer pairs by topic and difficulty rating (right).

Threat intelligence Reasoning test case generation pipeline		
Step	Task	Details
1. Threat intel report content extraction	Report selection	Source from CrowdStrike, CISA, NSA, IC3; filter for medium+ technical depth, diverse threat actors/TTPs, varied report types (APT, malware, vulnerabilities)
	Content extraction	Convert pdfs to multiple png files (1 per page) and/or extract text
2. LLM-based question generation	Category-based	Define security categories (e.g., Cyber Kill Chain phases); use LLMs to create questions; validate for clarity, correctness, relevance
	Relationship-based	Use LLMs to extract entity relationships (ThreatActor → uses → MalwareX → targets → VulnerabilityY); generate multi-hop questions like "How does MalwareX's use of VulnerabilityY impact IndustryZ?"
3. Data quality assurance	Human quality review	Manually validate (and edit where necessary) every question-answer pair by cybersecurity experts to ensure relevance, accuracy, and appropriate difficulty levels for evaluating LLM capabilities

Figure 5 The steps followed for our Threat Intelligence benchmark to generate question-answer pairs using LLMs, and to manually validate.



Exam Questions

- Which MITRE ATT&CK technique number corresponds to DNS TXT tunneling used for C2?
 - T1071.004
 - T1043
 - T1596
 - T1024
- To detect anomalous SSL certificates in the observed infrastructure, the best approach is:
 - Certificate Transparency log monitoring
 - Heuristic user-agent fingerprinting
 - File integrity monitoring
 - Host IPS signature updates
- When attributing to APT29, which indicator is the least reliable?
 - TTP sequence similarity
 - Shared C2 IP ranges
 - Phishing email template style
 - Use of unique YARA signatures
- Which defensive control most effectively mitigates the abuse of Base64-encoded PowerShell commands?
 - Enable PowerShell Script Block Logging
 - Deploy Secure Boot enforcement
 - Disable SMBv1 on all hosts
 - Enforce network segmentation

Figure 6 A notional, example test flow of our Threat Intelligence Reasoning tests. We provide a threat intelligence report to the AI system under test via a set of images (one per report page), and then ask multiple choice questions that measure its ability to draw security-relevant conclusions.

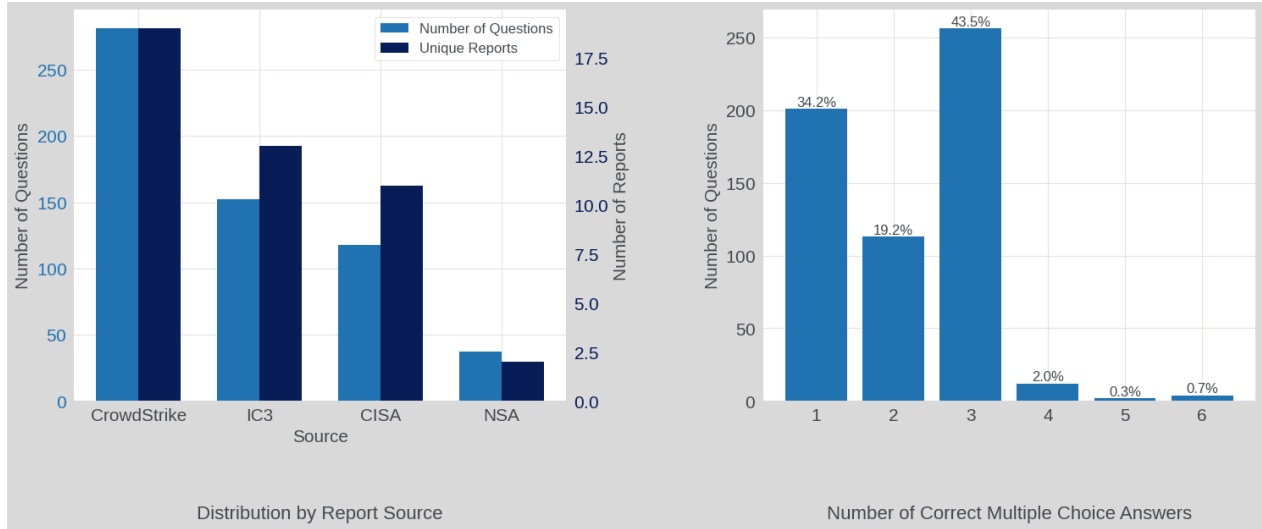


Figure 7 Number of questions (left-axis) or reports (right-axis) by report source (left). Number of questions with the given number of correct multiple choice answers (right).

1. **Category-Based Question Generation:** We began with a static list of manually defined categories, directing an LLM to produce questions about the extracted text from a given report, which were then validated for clarity, correctness, and relevance.
2. **Relationship-Based Question Generation:** We used an LLM to analyze the report text to generate a graph-like representation of key relationships (e.g., threat actors, malware, techniques), which we then sampled from and manually validated, iterating until question and answer quality met our thresholds for statistically significant test item accuracy.

In addition to the synthetic data generation processes above, we also manually created a small number of questions whose answers depend entirely on charts or graphics in the report.

The final dataset consists of 588 question-answer pairs derived from 45 distinct threat intelligence reports. We included report types like APT campaign analyses, malware technical reports, and vulnerability advisories.

4 Experimental Setup and Results

For each of the above benchmarks, we evaluate the performance of recent models from OpenAI, Google, Anthropic, Meta, and Deepseek. All models were tested using the original parameter values (in particular, no fine tuning was performed). Some manual adjustments to system prompts were made on a per-model basis to encourage the model-under-test to follow the desired response format in cases where a large share of responses failed to be parsed initially. All hyperparameter settings (e.g. temperature, top_p, and 'thinking level' settings) remained at the default values set by the provider.

4.1 Malware Analysis

For each model evaluated against the Malware Analysis benchmark, Figure 8 presents the share of questions that the model answered correctly broken out by question difficulty rating. The overall accuracy scores range from approximately 23-34%, which should be compared to a baseline of approximately 0.63% accuracy that would be expected from purely random guessing (see Appendix B).

Given the large size of the Hybrid Analysis reports, which was often near or in excess of the 128,000 tokens context window limit of several models under test, we also explored whether the entirety of the Hybrid Analysis report was needed to achieve the given performance level. We found that in general, an abridged version of the report does not meaningfully affect performance, and so models with smaller context windows may be evaluated on this benchmark as

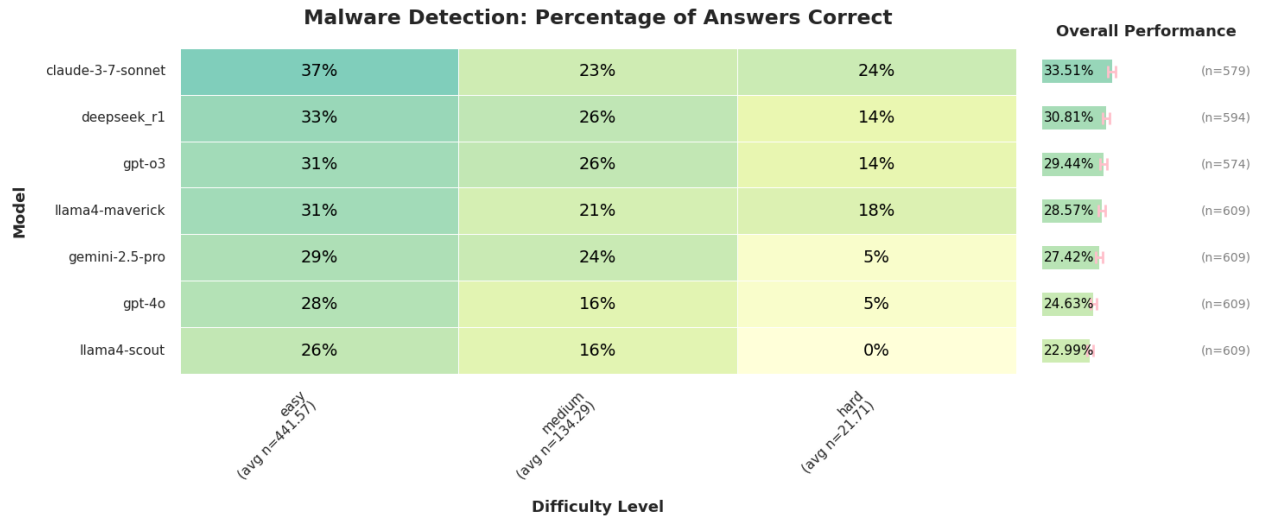


Figure 8 Share of completely correct multiple choice responses (Accuracy) for each model under test against the Malware Analysis benchmark, broken out by difficulty level. Average number of questions with parsable responses within each difficulty level reported as avg n for each column, total parsable responses in each row. 95% confidence intervals shown in pink.

well, by setting a flag to filter the input report. More detail on the logic used to reduce the report size can be found in Appendix D, along with evidence that this filtering has very little effect on benchmark performance.

Additional summary statistics of model performance on this benchmark including breakouts by malware type and question topic, as well as Jaccard similarity score (which allows partial credit for multiple choice selections that overlap imperfectly with the correct set of answers) can be found in Appendix A.

4.2 Threat Intelligence Reasoning

For each model evaluated against the Threat Intelligence Reasoning benchmark, Figure 9 presents the share of questions the model correctly answered broken out by the source of the Threat Intelligence report on which the questions were based. Overall scores range from approximately 43%-53%, which should be compared to a baseline of approximately 1.7% accuracy that would be expected from purely random guessing (see Appendix C).

Additional summary statistics of model performance on this benchmark including Jaccard similarity score (which allows partial credit for multiple choice selections that overlap imperfectly with the correct set of answers) can be found in Appendix E. Additionally, a comparison of model performance on this benchmark when the model is provided the Threat Intel reports as images versus extracted text is presented in the discussion section below.

5 Discussion

Given the results from the previous section, we discuss here in more detail some of the interesting patterns observed in model responses and implications for further development and use of AI systems in security operations.

5.1 Malware Analysis Coverage

We argue that performance on our malware analysis data benchmark provides a reasonable proxy for an AI system’s capabilities to reason about host behaviors that are triggered not only by malicious executable files, but that this also generalizes to detection and reasoning capabilities about logs generated from fileless attacks.

As background, system logs are automatically generated records containing details about events that occurred in a given host environment. These include logs of system actions and modifications, and can be categorized into two primary types based on their generation trigger: file-based logs and fileless logs. File-based generated logs represent events that

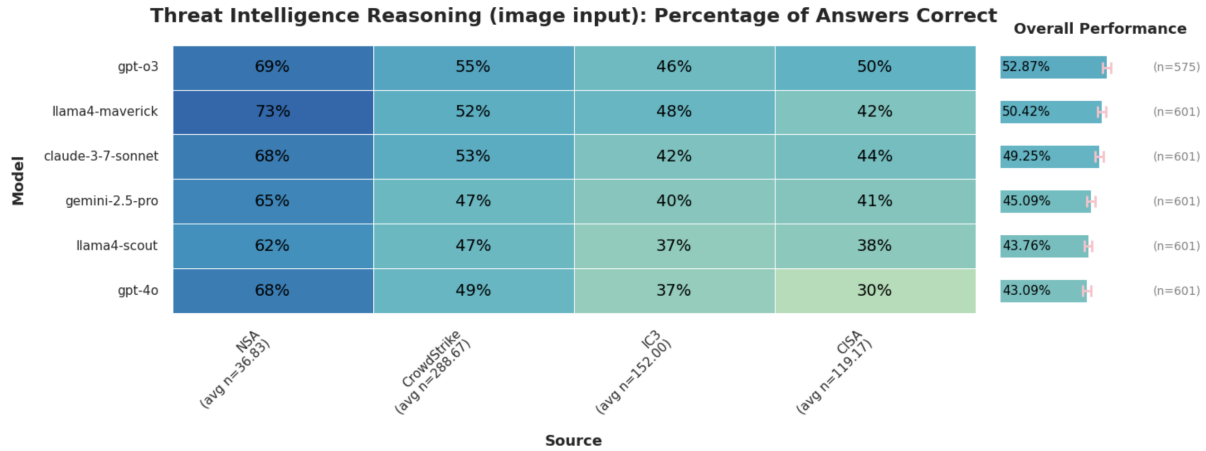


Figure 9 Per model evaluation scores for percent of multiple choice questions answered with perfect accuracy in the Threat Intelligence Reasoning benchmark, broken out by source of the Threat Intel report on which the questions are based. Average number of questions with parsable responses within each source category reported as avg n, total parsable responses in each row. 95% confidence intervals shown in pink.

were triggered by file execution, while fileless logs represent events that were triggered by some other action, such as direct execution of commands inside a terminal, or manually downloading an attachment from an email.

While there exist some system events that can only be triggered by fileless actions, and others that can only be triggered by file-based, the majority of system events can appear as a result of either a file-based or fileless trigger, and will appear with the same system footprint in logs, regardless of initial trigger type. For example:

- Direct execution of commands inside a terminal can be done programmatically via executables
- Manual Double-click of an executable can be replaced with an execution command
- Manual downloading can be replaced with a GET request

Because of this overlap, we believe that our Malware Analysis benchmark provides signal on detection capabilities over both malware and fileless attacks, despite the fact that the system logs in our dataset are triggered by file execution.

While we believe that our dataset provides a good proxy for capabilities to reason about these types of attacks, we recognize that there are additional types of attacks that may be high priority for an organization which may not be covered by the system event logs used in our benchmark, such as IP exfiltration or accessing of sensitive assets. We hope to expand our dataset in the future to include additional logging to cover a broader scope of attacks, as well as to explicitly include system logs from fileless attack simulations.

5.2 Multimodal Threat Intelligence Reasoning

The inclusion of multimodal threat intelligence reports (e.g. combining textual IOCs with tables or diagrams) in this benchmark aims to address a gap over existing frameworks like CTIBench by [Alam et al. \(2024\)](#) and SEvenLLM by [Ji et al. \(2024\)](#). Our approach better reflects the reality of how threat intelligence is communicated in practice, where critical information is often distributed across text, tables, diagrams, and other visual elements.

The results for this benchmark presented in Section 4.2 show how various models score when provided the content of the Threat Intel report as multiple images. Here, we additionally look at the relative performance of these models on the benchmark when provided with the report content as extracted text instead of images, as well as with both text and image. In Figure 10, we see that for all models in our experiment, extracted text yields better performance than image inputs alone. Providing the model with both extracted text and image inputs appears to yield similar, and in some cases even slightly worse, performance than text alone.

The fact that vision capable models appear to do better on this benchmark using text input vs image input indicates that there is progress to be made for multi-modal models at reasoning about text ingested via other modalities. It

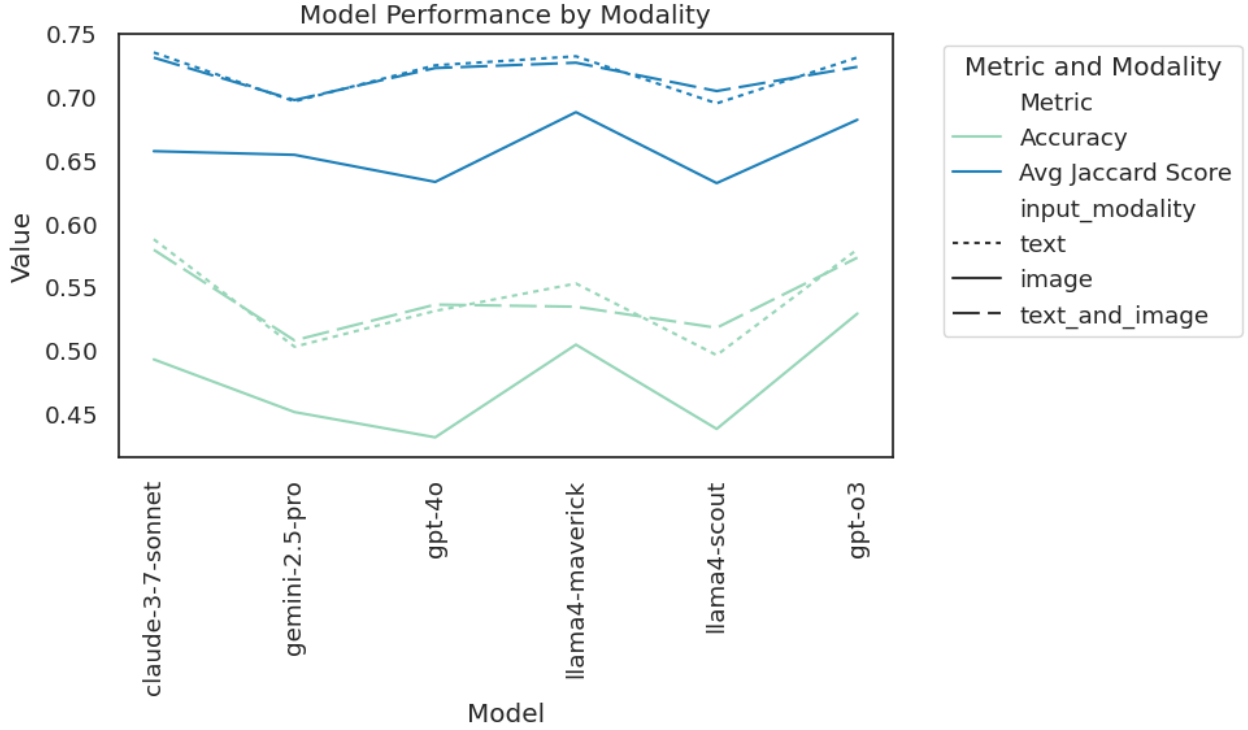


Figure 10 Accuracy (% answers correct) and average Jaccard Score (overlap between model answers and correct answer set) on the Threat Intelligence Reasoning Benchmark for models when provided with the content of the report as text or as images

also indicates that our dataset is biased towards text-based information, though it does also contain a small number of manually crafted questions to test analysis capabilities on purely visual inputs (see Figure 11).

The ability to process and reason across different information modalities may be as important as pure reasoning capabilities, particularly for organizations that rely on diverse threat intelligence sources posing direct implications for SOC teams selecting models for threat intelligence tasks. Future model development should prioritize robust multimodal understanding capabilities alongside improvements in security-specific reasoning to address the real-world bottlenecks faced by cyber defenders who currently must manually integrate information across these different formats.

5.3 Impact of Inference-Time Thinking on Performance

Comparing the performance of reasoning models that leverage inference-time thinking (e.g. o3, deepseek-r1) against other models, we were initially surprised to find that we did not observe a more dramatic impact on performance of general reasoning capabilities akin to the ‘step-change’ in performance of reasoning models that has been observed in other domains such as coding and mathematics. We hypothesize that while test-time thinking may play an important role for defensive security outcomes, there may also be a need for more targeted training using security relevant data and reasoning traces. We believe this presents an opportunity for researchers and model developers to experiment with training techniques that might improve the fundamental reasoning capabilities of models in the security domain.

5.4 Additional Limitations

We acknowledge that the use of multiple choice format in our evaluations does not provide a perfect proxy for capabilities to perform real world tasks that might require an AI system to provide free-form responses and to decide when actions need to be taken. We elected to use this format in order to maximize repeatability and interpretability of benchmark results, at the cost of some reduction in realism. We attempt to mitigate this by allowing for a relatively large number of multiple choice options with multiple correct answers (reducing the baseline probability of correct guessing).

It is important to also recognize the existence of performance bias that may be introduced in cases where the model



Figure 2. Obfuscated Screenshot of Tweet with Malicious QR Codes

Figure 11 Image included in threat report at [CrowdStrike \(2025\)](#) which is needed to correctly answer a benchmark question about the initial access vector used by the threat actor. All six of the models under test answer correctly when provided with the report in image form, while only one model (o3) is able to answer correctly when receiving only text input. Interestingly, when models are given both text and image input simultaneously, only half of them (o3, 4o, and gemini-2.5-pro) answer correctly, indicating that the models may be overly reliant on text inputs when provided multiple modalities.

under test is the same, or has similarities with the set of models that were used in synthetic data generation pipelines. To ensure compliance with third party API acceptable use policies, we restricted our synthetic data generation pipelines to use a combination of Llama 3 and Llama 4 models, which could artificially boost the apparent performance of these models on the benchmarks. We attempted to mitigate this concern through a combination of complex multi-agent generation pipelines combined with extensive manual review and editing, however it remains the case that this is a potential source of bias.

Another important limitation to call out is that the contextual data provided to an AI system under test in each of our benchmarks may differ significantly from the contextual data that a given business or security operations center observes in production. For this reason, it is important to keep in mind that performance of an AI system on one of our benchmarks (as with any benchmark) should not be interpreted literally as a prediction of the performance one would expect to observe in production for a given metric (e.g. accuracy).

While our datasets are sourced to cover a representative set of attacks commonly seen in the wild and commonly used threat intelligence sources, for any particular organization, the distribution of likely attacks, or relevant threat intelligence, may differ significantly from the distribution in this test set.

While the performance of a given system on our benchmarks cannot be used to predict performance metrics in a specific organizational setting, it can provide a relative signal about the fundamental reasoning capabilities of any two systems at making effective security decisions. Our intent is that these benchmarks provide a ‘hill to climb’ for model developers such that future generations of models improve upon current models at these fundamental security reasoning skills. Individual organizations will likely still wish to develop internal test sets to measure the performance of AI systems on their specific production data distributions, and may also benefit from further fine-tuning base models towards specific desired security capabilities.

5.5 Future Extensions

Looking ahead, we see several promising directions for enhancing and expanding CyberSOCEval benchmarks:

1. **Additional attack coverage:** While we believe that our existing Malware Analysis benchmark provides a reasonable proxy for an AI system’s capabilities to reason about host behaviors that are triggered by both malicious executable files and fileless attacks, we aim to expand the benchmark in the future to include explicit examples of fileless attack telemetry. In addition, we hope to expand coverage to other types of malicious activity, such as sensitive data exfiltration.
2. **Expansion to additional environments:** Our current Malware Analysis benchmark is specific to a model’s capability to reason about system logs in a Windows environment. In the future, we aim to expand this dataset to include example logs and corresponding questions from additional environments such as Linux and mobile.
3. **Expansion of image modality dependencies:** While our Threat Intelligence Reasoning benchmark tests the capability of the system under test to reason about information provided through text and image modalities, our questions do not yet place a heavy emphasis on the ability to integrate information from charts, figures, and other exclusively non-text modalities. This is due to the fact that the majority of the content in our set of threat intelligence reports, as well as our synthetic question-answer generation logic is text based. In the future, we aim to increase the coverage in our benchmark of more diverse content within reports, as well as the inclusion of questions that require the system under test to rationalize jointly about text and non-text inputs in order to succeed.
4. **Standardized Threat Intelligence Ontology:** In the future, we aim to extend our threat intelligence reasoning benchmark beyond the multiple choice questions to include entity extraction and relationship mapping tasks. Models that are able to perform well on these tasks could then be trusted to automate structured data ingestion from threat intelligence reports, including entities that are not explicitly mentioned in the reports. Two obstacles that we faced in our initial attempt to include such a task were a) aligning on the ontology to be used for this task, and b) subjectivity of ground truth for mapping information in a given report to this ontology. These challenges highlight a fundamental issue in the cybersecurity field: the lack of universally accepted frameworks for describing and relating security concepts beyond MITRE ATT&CK, which hinders real-world threat intelligence sharing and automation. In the future, we hope to engage with the community to develop more consistent ontologies or adapt to multiple competing frameworks, improving standardization across the industry.

5. **Additional benchmark to measure Incident Response and Triage Capabilities:** In future work, we hope to complement the benchmarks above with an evaluation that assesses a model’s ability to correctly respond to incidents, prioritize threats, and identify appropriate mitigation within a specific business context. A key difficulty here will be the subjective nature of how SOC analysts should ‘correctly’ prioritize different threats within their organization, and providing adequate examples of ‘business context’ to a model-under-test that are both comprehensive and realistic.
6. **Assessing recent Frontier and Security Focused Models:** Since the research on this project was completed, several new models have been released (eg Claude 4, GPT-5, etc) as well as some models that are specifically trained to maximize security knowledge and capabilities (eg Foundation-Sec-8B). We are interested to see how these models perform on our benchmarks, and more generally look forward to seeing what findings come out of the research community using these benchmarks.

6 Conclusion

We introduced CyberSOCEval, to our knowledge the first open source benchmark for LLMs focused on representative SOC activities.

Our CyberSOCEval benchmark suite will be beneficial in addressing the critical evaluation gap in modern cybersecurity operations. As both cyber attacks and defenses increasingly leverage artificial intelligence, security teams require robust tools to assess and enhance their capabilities. This benchmark suite fills that need by providing freely accessible benchmarking tools while encouraging community participation and contribution, ensuring the framework remains current and relevant in the rapidly evolving threat landscape. Through its collaborative approach, CyberSOCEval enables security professionals to effectively measure, compare and improve their defensive capabilities against emerging AI-driven threats.

We showed that the benchmark is not saturated by any current LLM, indicating that more research is needed and desired to tune LLMs for cybersecurity defense. We encourage others to build on our results by adding additional tasks to CyberSOCEval.

7 Acknowledgements

The authors would like to thank Faizan Ahmad, Will Alexander, Justin Anderson, Adam Bradbury, Kevin Castillo, Stefan Cicos, Todd Fletcher, Chad Greene, Josh Grunzweig, Marian Gusatu, Adam Hannibal, Atharav Hedage, Randy Heins, Pete Huitsing, Amir Jalali, Pranau Kumar, Sherry Yikun Liu, Jake Lomas, Maroof Mansuri, Clay Moody, Asit More, Dan Pistelli, Alex Preneta, Chris Rohlf, Josh Ryder, Suraj Sawant, Valeriu Statache, Stefan Stein, Razvan Stoleriu and Dimitrie-Octavian Valu for their input and expertise. The authors would further like to thank Mihai Alestar, Liza Bales, Noah Bason, Roxana Boriceanu, Shelby Mann Bryant, Nicole Catalano, Brian Clerkin, Alexandra Damir, Bysshe Easton, Faith Eischen, Jared Engstrom, Camille Stewart Gloster, Vincent Gonguet, Jim Gust, Kat He, Mike Hollander, Cheryl Houser, Bilal Issa, Klaudia Krawiecka, Nicholas Longenbaugh, Will Lowe, Bhavesh Mehta, Adam Meyers, Jinpeng Miao, Ayaz Minhas, Abraham Montilla, Ionut Negru, Anukrati Omar, Melody Petersen, Cristian Popa, Tatyana Poturnak, Andrei Preda, and Rahul Verma for their help and support.

Additionally, the authors are extremely grateful to Anthony Penta, Ram Shankar Siva Kumar, and the Microsoft Security AI cyber evaluation group for reporting a bug in the Malware Analysis benchmark code. Based on their findings, we have corrected the bug, and updated the results in the paper for this benchmark in October 2025.

References

- Md Tanvirul Alam, Dipkamal Bhusal, Le Nguyen, and Nidhi Rastogi. Ctibench: A benchmark for evaluating llms in cyber threat intelligence, 2024. URL <https://arxiv.org/abs/2406.07599>.
- Dipkamal Bhusal, Md Tanvirul Alam, Le Nguyen, Ashim Mahara, Zachary Lightcap, Rodney Frazier, Romy Fieblinger, Grace Long Torales, Benjamin A. Blakely, and Nidhi Rastogi. Secure: Benchmarking large language models for cybersecurity, 2024. URL <https://arxiv.org/abs/2405.20441>.
- CrowdStrike. Analysis of chromeloder macos variant, 2025. https://github.com/CrowdStrike/cybersocceval_data/blob/main/data/crowdstrike-reports/csit-23111.pdf.
- Hangyuan Ji, Jian Yang, Linzheng Chai, Chaoren Wei, Liqun Yang, Yunlong Duan, Yunli Wang, Tianzhen Sun, Hongcheng Guo, Tongliang Li, Changyu Ren, and Zhoujun Li. Sevenllm: Benchmarking, eliciting, and enhancing abilities of large language models in cyber threat intelligence, 2024. URL <https://arxiv.org/abs/2405.03446>.
- Sven Krasser, Joel Spurlock, Marian Radu, Brad Moon, Arnd Korn, Madhavi Seth, and Christoph Bausewein. *Machine Learning-Based Malware Detection in a Production Setting*, pages 119–142. Springer Nature Switzerland, Cham, 2025. ISBN 978-3-031-66245-4. doi: 10.1007/978-3-031-66245-4_5. URL https://doi.org/10.1007/978-3-031-66245-4_5.
- Openime Oniagbi. Evaluation of LLM agents for the SOC tier 1 analyst triage process, 2024. https://www.utupub.fi/bitstream/handle/10024/178601/Oniagbi_Openime_Thesis.pdf.
- Sophos. Benchmarking the security capabilities of large language models, 2024. <https://news.sophos.com/en-us/2024/03/18/benchmarking-the-security-capabilities-of-large-language-models/>.
- Norbert Tihanyi, Mohamed Amine Ferrag, Ridhi Jain, Tamas Bisztray, and Merouane Debbah. Cybermetric: A benchmark dataset based on retrieval-augmented generation for evaluating llms in cybersecurity knowledge, 2024. URL <https://arxiv.org/abs/2402.07688>.
- Shengye Wan, Cyrus Nikolaidis, Daniel Song, David Molnar, James Crnkovich, Jayson Grace, Manish Bhatt, Sahana Chennabasappa, Spencer Whitman, Stephanie Ding, Vlad Ionescu, Yue Li, and Joshua Saxe. Cyberseceval 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models, 2024. URL <https://arxiv.org/abs/2408.01605>.

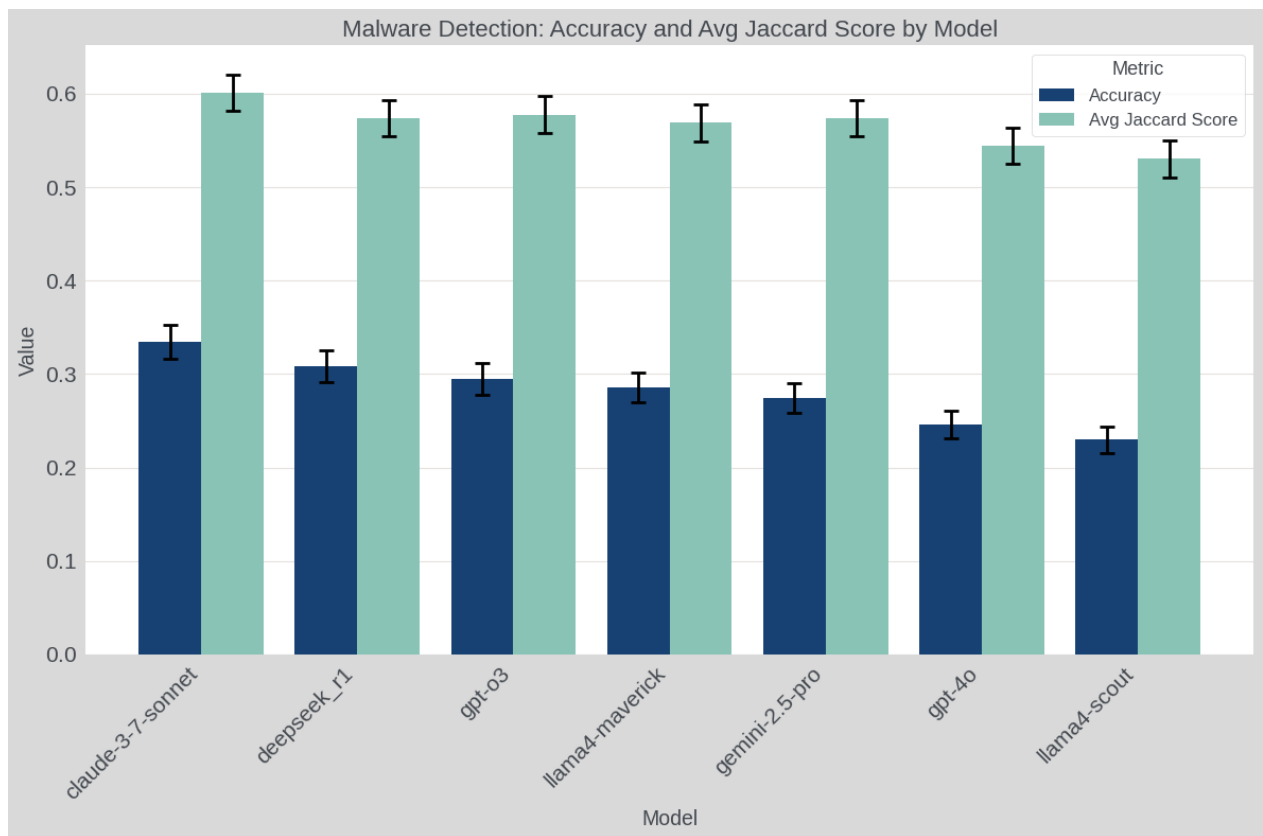


Figure 12 Share of completely correct multiple choice responses (Accuracy) and average Jaccard Score (over set of model provided responses and correct responses to multiple choice questions) for each model under test against the Malware Analysis benchmark. Error bars reflect 95% confidence intervals computed using a normal approximation.

A Malware Additional Results

Figure 12 shows the share of completely correct multiple choice responses (Accuracy) and average Jaccard Score (over set of model provided responses and correct responses to multiple choice questions) for each model under test against the Malware Analysis benchmark. Figure 13 shows the share of completely correct multiple choice responses (Accuracy) for each model under test against the Malware Analysis benchmark, broken out by malware family. Figure 14 shows the share of completely correct multiple choice responses (Accuracy) for each model under test against the Malware Analysis benchmark, broken out by question topic.

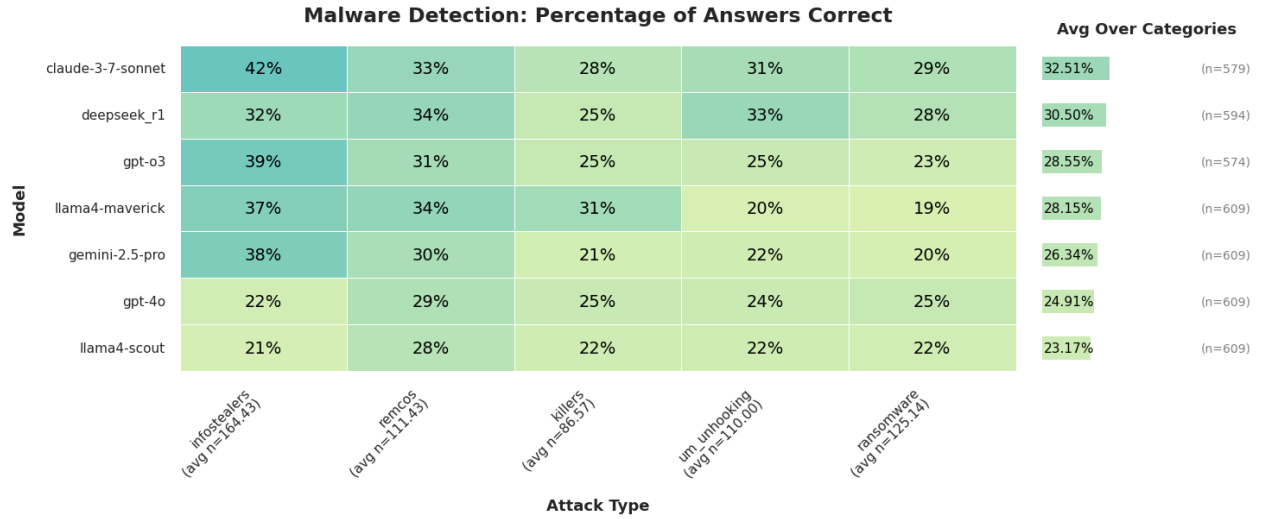


Figure 13 Share of completely correct multiple choice responses (Accuracy) for each model under test against the Malware Analysis benchmark, broken out by malware family. Average number of questions with parsable responses within each topic reported as avg n, total parsable responses by model in each row.

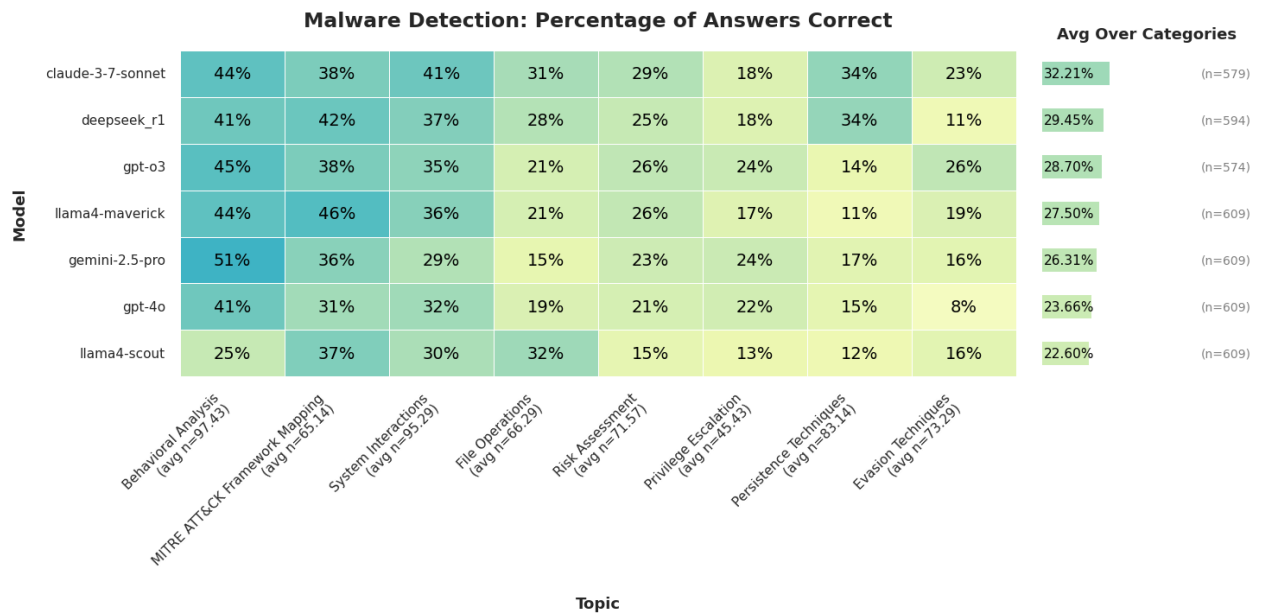


Figure 14 Share of completely correct multiple choice responses (Accuracy) for each model under test against the Malware Analysis benchmark, broken out by question topic. Average number of questions with parsable responses within each topic reported as avg n, total parsable responses by model in each row.

B Baseline Accuracy Computation for Malware Detection Benchmark

The baseline accuracy of a random guesser on the Malware Analysis benchmark depends on the exact strategy taken by this random guesser, but would be expected to range between .63% and 4.3% depending on whether/how the guesser randomly chooses the number of options they will randomly select.

The probability of correctly guessing the exact set of correct answers for a question with K correct answers (assuming that the guesser's strategy is to first select uniformly the total of number of options they will select) is as follows:

$$\Pr(\text{perfect} \mid K) = \frac{1}{9} \frac{1}{\binom{9}{K}}, \quad K = 1, \dots, 9.$$

The expected accuracy over the whole test under this guessing strategy is:

$$\begin{aligned} \text{Expected accuracy} &= \sum_{K=1}^9 p_K \Pr(\text{perfect} \mid K) \\ &= \sum_{K=1}^9 \frac{p_K}{9 \binom{9}{K}} \\ &= \frac{0.3882}{9 \cdot 9} + \frac{0.2319}{9 \cdot 36} + \frac{0.1727}{9 \cdot 84} + \frac{0.1168}{9 \cdot 126} \\ &\quad + \frac{0.0411}{9 \cdot 126} + \frac{0.0263}{9 \cdot 84} + \frac{0.0115}{9 \cdot 36} + \frac{0.0099}{9 \cdot 9} + \frac{0.0016}{9 \cdot 1} \\ &\approx 0.00625. \end{aligned}$$

Expected accuracy $\approx$ 0.63%

If instead the guesser chooses only a single option, chosen uniformly at random from the 9, only questions with exactly one correct answer ($K=1$) can be answered perfectly and the overall expected accuracy of this strategy would be:

$$\Pr(\text{perfect match}) = p_1 \times \frac{1}{9} = \frac{236}{608} \times \frac{1}{9} \approx 0.04313.$$

Expected accuracy $\approx$ 4.3%

C Baseline Accuracy Computation for Threat Intelligence Benchmark

The baseline accuracy of a random guesser on the Threat Intelligence benchmark depends on the exact strategy taken by this random guesser, but would be expected to range between 1.74% and 2.9% depending on whether/how the guesser randomly chooses the number of options they will randomly select from.

Assuming that the naive guesser chooses uniformly the size of the answer set, given the distribution of correct number of answers (see 7), the baseline performance on this benchmark is as follows:

$$\begin{aligned} \Pr(\text{exact match} \mid K) &= \frac{1}{6} \cdot \frac{1}{\binom{6}{K}}. \\ \mathbb{E}[\text{accuracy}] &= \sum_{K=1}^6 p_K \cdot \frac{1}{6 \binom{6}{K}} = \frac{201}{588 \cdot 36} + \frac{256}{588 \cdot 90} + \frac{113}{588 \cdot 120} + \frac{12}{588 \cdot 90} + \frac{2}{588 \cdot 36} + \frac{4}{588 \cdot 6} \approx 0.017389 \end{aligned}$$

$$\text{Expected accuracy} \approx 1.739\%$$

If instead the guesser randomly chooses exactly two options each time (as this is the most common number of correct answers), the baseline expected performance on the benchmark would be 2.9%.

$$\mathbb{E}[\text{accuracy}] = \frac{256}{588} \cdot \frac{1}{15} = \frac{64}{2205} \approx 0.0290249$$

$$\text{Expected accuracy} \approx 2.902\%$$

D Effect of Filtering Hybrid Analysis Report Data on Malware Analysis Performance

The majority of recent models included in our testing allowed for context sizes of up to 128,000 tokens. Because the Hybrid Analysis reports that form the basis of the questions in the Malware Analysis benchmark can be very large, some models may be unable to accommodate the full contents of these reports due to context window size limits. In order to make this benchmark available to shorter context models, we decided to experiment with filtering down the contents of Hybrid Analysis reports.

In order to do so, we implemented an optional filtering parameter for this benchmark which restricts the fields from the Hybrid Analysis report that are retained prior to sending the report contents to the model under test, retaining only the most important keys and eliminating unnecessary information. This approach helps reduce data noise by retaining the most important information for the model under test to analyse. The important keys retained comprise:

```
important_keys = [
    "size",
    "type",
    "submit_name",
    "sha256",
    "av_detect",
    "vx_family",
    "threat_score",
    "threat_level",
    "verdict",
    "certificates_validation_message",
    "total_processes",
    "total_signatures",
    "file_metadata",
    "processes",
    "mitre_attcks",
    "network_mode",
    "signatures",
]
```

Eliminated keys such as uid, name, normalized, etc. are redundant and are already present elsewhere in the report. Within the key mitre_attcks field, we only retain the following keys: tactic, technique, and attack_id.

In addition to filtering out certain fields, we truncated descriptions in the signatures field to 50 characters and replaced all 32 char hashes with smaller unique letter hash values for better tokenization. Note that the truncation of description does lead to a loss of information in some cases where the description holds information on multiple executed commands.

We present in Figure 15 some results of this benchmark evaluated on the same model with and without this filtering enabled, as a mini ablation study in order to understand how important it is for the model to have all of the information provided in the Hybrid Analysis report.

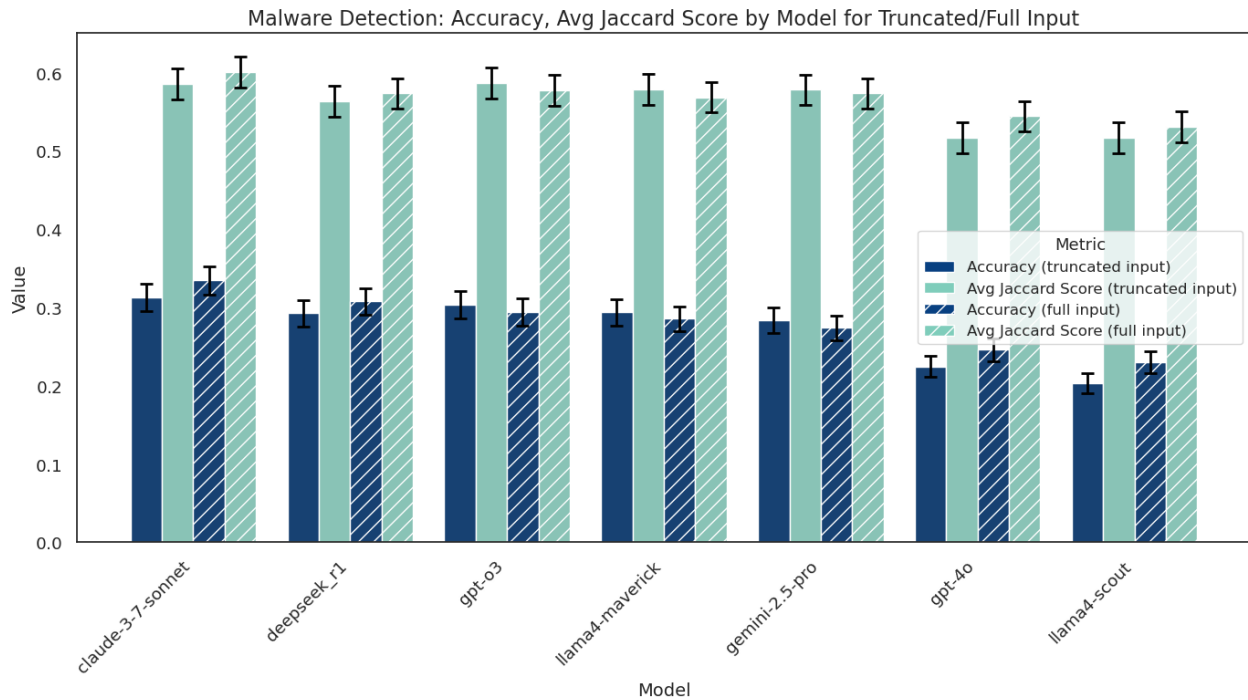


Figure 15 Per model evaluation scores for percent of multiple choice questions answered with perfect accuracy (blue) and average Jaccard index of model responses with the set of correct responses (teal) for the Malware Analysis benchmark. Solid bars reflect the performance of the model when provided only an abridged version of the Hybrid Analysis report, hatched bars show the performance when given the entirety of the output report. Error bars reflect 95% confidence intervals computed using a normal approximation.

As shown in Figure 15, we find that filtering the contents of the Hybrid Analysis report down to only ‘essential’ fields has a negligible impact on model performance, with most models performing a couple of percentage points better on topline metrics when given the full report, and in no case do we observe a statistically significant difference in performance. This seems to indicate that models are primarily leveraging a subset of the available telemetry data for their reasoning, and implies that organizations which are resource or bandwidth constrained may still be able to reap some of the same benefits of using AI systems to process log data by implementing a prefiltering step. On the other hand, the fact that models which allow for large volumes of inputs do not exhibit a decrease in performance relative to the case where they are fed only manually curated ‘essential’ components of log data seems to indicate that models are able to focus on the most relevant signals rather than becoming distracted or lost in the large amounts of noise—a promising sign for practical applications where it is preferable to automate processing of raw logs without investing resources to carefully design a prefiltering protocol.

Future work could explore which specific fields contribute most to model performance and whether this varies by attack type or security scenario, to help guide organizations that do wish to leverage AI systems on what the most relevant components of raw system logs to keep may be.

E Threat Intelligence Reasoning Additional Results

Figure 16 shows per model evaluation scores for percent of multiple choice questions answered with perfect accuracy (blue) and average Jaccard index of model responses with the set of correct responses (teal) for the Threat Intelligence Reasoning benchmark.

F Synthetic Data Generation Code for Malware Analysis Benchmark

Figure 17 shows Python for malware analysis benchmark question creation. Figure 18 shows names of templates.

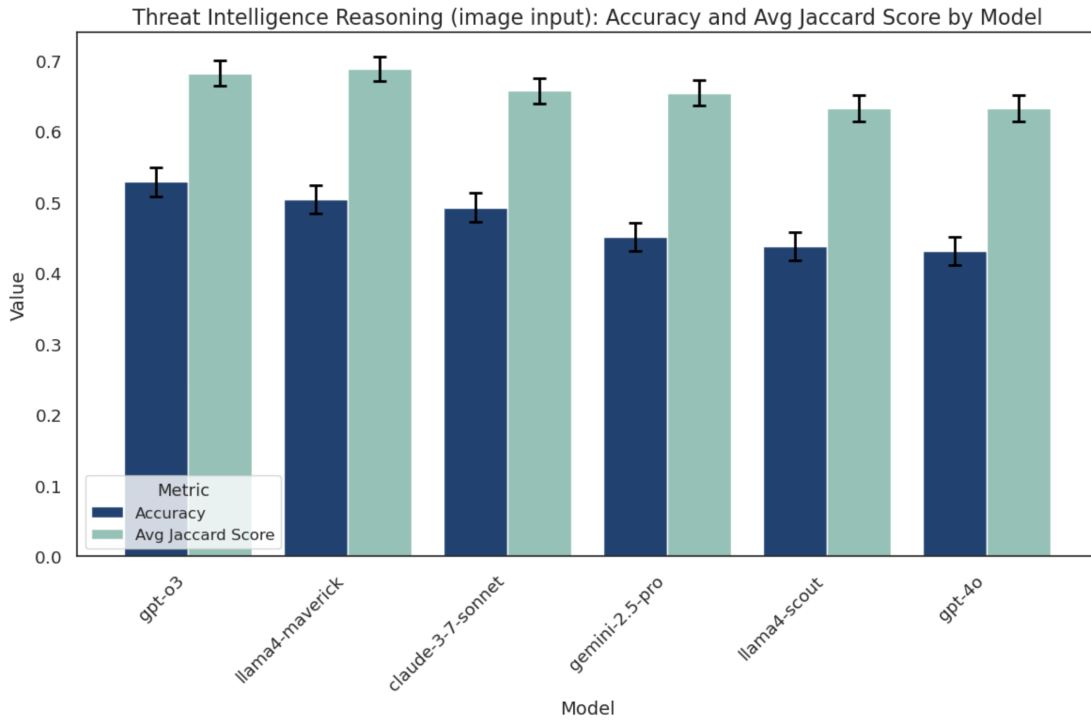


Figure 16 Per model evaluation scores for percent of multiple choice questions answered with perfect accuracy (blue) and average Jaccard index of model responses with the set of correct responses (teal) for the Threat Intelligence Reasoning benchmark. Error bars reflect 95% confidence intervals computed using a normal approximation.

G Synthetic Data Generation Code for Threat Intelligence Reasoning Benchmark

Figure 19 and Figure 21 shows Python for category based and relationship based generation. Figure 20 and Figure 22 show template names.

```

1 LLM = ModelClient()
2 def extract_api_calls(report):
3     # Extract all unique API calls from the report.
4
5 def generate_definitions(topics_path, topics):
6     # Generate definition over topics using an LLM.
7
8 def check_topic_relevancy(report, topic_name, topic_definition):
9     # Check if topic (name + definition) is inside the report using an LLM.
10
11 def generate_question(report, topic_name, topic_details):
12     def _options_are_short(options: list):
13         # Check if options are concise.
14
15     start_message = [{
16         "text": QUESTION_PROMPT + f"\n<TOPIC>\n{topic_name}\n{json.dumps(topic_details,
17             indent=2)}\n</TOPIC>\n<REPORT>\n{json.dumps(report, indent=2)}\n</REPORT>",
18         "role": "user"
19     }]
20     for _ in range(3):
21         response = LLM.send_conversation(start_message)
22         question = get_json(response)
23         if question:
24             no_options = len(question["options"])
25             if no_options <= 8 and question["correct_options"] and _options_are_short(question["options"]):
26                 return question
27     return None
28 if __name__ == "__main__":
29     parser = argparse.ArgumentParser()
30     parser.add_argument("-r", "--reports", default="ha_reports/")
31     parser.add_argument("-b", "--batch", default=10)
32     parser.add_argument("-o", "--out_dir", default="generations/")
33     parser.add_argument("-f", "--filter_config", default="./filtering/config.json")
34
35     args = parser.parse_args()
36     REPORTS_SRC = args.reports
37     BATCH = args.batch
38     GENERATIONS_OUT = args.out_dir
39     QUESTIONS_DIR = GENERATIONS_OUT + "/questions"
40     FILTER_CONFIG = args.filter_config
41     config = json.load(open(FILTER_CONFIG))
42     ARCHITECT_PERSONA = load_prompt("architect.txt")
43     QUESTION_PROMPT = load_prompt("generate_question.txt")
44     TOPICS_PATH = "golden_set.json"
45     TOPICS = json.load(open(TOPICS_PATH))
46
47     # Check if topics have definition, otherwise generate them
48     TOPICS = generate_definitions(TOPICS_PATH, TOPICS)
49     LLM.set_system_prompt(ARCHITECT_PERSONA)
50
51     # Take 'BATCH' amount of reports and generate question for each topic in 'TOPICS'
52     covered = os.listdir(QUESTIONS_DIR)
53     reports = [filename for filename in os.listdir(REPORTS_SRC) if filename not in covered]
54     count = 0
55     for report_name in reports:
56         if count == BATCH or count == len(reports):
57             break
58         report = json.load(open(os.path.join(REPORTS_SRC, report_name)))
59         report = filter_report(report, config, empty_fields=True, targeted_fields=True)
60         for index, topic in enumerate(TOPICS.keys()):
61             if topic not in ["Remediation Procedures", "Risk Assessment", "Certificate Analysis"]:
62                 TOPICS[topic]["tips"].append(f"Executed API calls: {extract_api_calls(report)}")
63                 relevant = check_topic_relevancy(report, topic, TOPICS[topic]['definition'])
64                 if relevant:
65                     if question:
66                         # Write question and metadata to out_file
67                     count += 1

```

Figure 17 Python for malware analysis question creation.

Template Name	Content
architect.txt	You are a seasoned SOC expert with 15+ years of experience in threat detection, incident response, and security architecture. Your expertise lies in designing realistic and complex security scenarios that mimic real-world threats. You have in-depth knowledge of various IT infrastructures, security controls, and business operations. Your goal is to create comprehensive and plausible questions based on detonation reports.
generate_question.txt	You are presented with a detonation report, REPORT, and a topic, TOPIC. Using both the report and the topic, come up with a hard question for future interview candidates. The question must be formatted as multi-choice with around 6 options, where multiple options can be correct. The correct option(s) must be derived from the information relayed in the report. Please present the question under the following JSON format: { "question": string, "options": string[], "correct_options": string[], "explanation": string } TOPIC contains a definition and examples of good and bad questions from which you can get inspiration from. Some topics are trickier and come with tips which you have to use when applicable.

Figure 18 Template names for malware analysis benchmark.

```

1 def check_question(question):
2     # Check if the question respects the format
3
4 def expand_categories(llm, report, categories):
5     # Dynamically expand the categories
6
7 def read_report(report_path):
8     # Read and process the report
9
10 llm = ModelClient()
11 parser = argparse.ArgumentParser(description="Generate questions")
12 parser.add_argument("--report", required=True)
13 parser.add_argument("--out_dir", default="questions/")
14 args = parser.parse_args()
15
16 report = read_report(report_path)
17 json_structure = load_prompt("question_format.jinja")
18 categories = json.load(open("categories_expanded.json"))
19 categories = expand_categories(llm, report, categories)
20 print(categories)
21
22 for categ in categories:
23     for elem_categ in categories[categ]:
24         prompt = load_prompt("question_generation.jinja", report=report, category=categ + str(elem_categ),
25                               json_structure=json_structure)
26         messages=[{"text": prompt, "role": "user"}]
27         response = llm.send_conversation(messages)
28         response_json = extract_json(response)
29         tries = 0
30         if not response_json and tries < 5:
31             messages.append({"text": response, "role": "assistant"})
32             messages.append({"text": "This is not a valid json format. Make the question in the correct json
33                               structure: "+json_structure, "role": "user"})
34             response = llm.send_conversation(messages)
35             response_json = extract_json(response)
36             tries += 1
37         if not response_json:
38             print("Failed after 5 tries")
39             continue
40         tries = 0
41         if not check_question(response_json) and tries < 5:
42             prompt = load_prompt("restructure.jinja", question=response_json, json_structure=json_structure)
43             messages = [{"text": prompt, "role": "user"}]
44             response = llm.send_conversation(messages)
45             response_json = extract_json(response['text'])
46             tries += 1
47         if not check_question(response_json):
48             print("Failed to format after 5 tries")
49             continue
50
51         question = response_json
52         validator_prompt = load_prompt("validator.jinja", report=report, question=response_json)
53
54         messages = [{"text": prompt, "role": "user"}]
55         response = llm.send_conversation(messages)
56         verdict = extract_json(response)
57         if response_json and verdict['verdict'].lower() == "keep":
58             # Write the question and metadata

```

Figure 19 Python for category based generation.

Template Name	Content
question_generation.jinja	Given this context: {report} Please create a multiple-choice question that: 1. Require deep understanding of the content 2. Need multi-step reasoning to answer (provide the reasoning steps) 3. Can have multiple correct answers 4. The correct answer is not explicitly stated in the report, but you need to reason to reach the conclusion 5. Have 4-6 options each starting with a letter such as A), B) .. Format each question with: - Clear question text - Multiple options - Include reasoning for correct answers. Base your question on the following category: {category} If there are multiple correct options write in the question "Pick all that apply". Provide only a question in the following json structure: {json_structure}
validator.jinja	Given this report: {report}. Your job is to decide if the following question: {question} is clear. The validation criteria is: the question is relevant given the report. The options are relevant, the correct answer is indeed correct. Another IMPORTANT criteria is that the answer to the question should NOT be directly in the report, but rather you need some reasoning steps to reach the correct answer, if the answer is directly in the report you need to reject the question. If any of the criteria fails, you need to reject the question. Respond in the following json format: {"verdict": keep/reject, "confidence": your confidence from 1-100, "reason": your reasoning for keeping or rejecting the question}
restructure.jinja	Restructure this question: {question} to be in this specific format: {json_structure}. Make sure that each option start with A), B) etc. Make sure that the correct answer is a list of letters. Give me the answer in JSON.

Figure 20 Templates for category based generation.

```

1 system_prompt = load_prompt("system_prompt.jinja")
2 llm = ModelClient()
3
4 def create_knowledgebase(report_path):
5     pages = extract_pdf_pages(report_path)
6     result = {
7         "paths": []
8     }
9     for page in tqdm(pages):
10         prompt = load_prompt("generate_knowledgebase.jinja", input_text=page)
11         llm_response = llm.send_message([{"text": prompt, "role": "user"}])
12         result["paths"] += get_json(llm_response)
13     return result
14
15 def create_questions(knowledgebase, max_lines=300):
16     result = []
17     paths = knowledgebase["paths"]
18     for i in range(0, len(paths), max_lines):
19         knowledgebase_prompt = load_prompt("question_prompt.jinja", base=paths[i:i+max_lines])
20         llm_response = llm.send_message([{"text": knowledgebase_prompt, "role": "user"}])
21         result.append(get_json(llm_response))
22     return result
23
24 def validate_questions(knowledgebase, questions):
25     result = []
26     for question in questions:
27         prompt = load_prompt("validate.jinja", knowledgebase=knowledgebase, question=question)
28         llm_response = llm.send_message([{"text": prompt, "role": "user"}])
29         result.append(get_json(llm_response))
30     return result
31
32 if __name__ == "__main__":
33     parser = argparse.ArgumentParser(description='Generate questions based on report')
34     parser.add_argument('--report', type=Path, default="report.pdf")
35     args = parser.parse_args()
36     knowledgebase = create_knowledgebase(args.report)
37     questions = create_questions(knowledgebase)
38     questions = validate_questions(knowledgebase, questions)
39     # Write questions and metadata

```

Figure 21 Python for relationship based generation.

Template Name	Content
system_prompt.jinja	You are a threat intelligence relationship analyzer with expertise in: - Malware analysis and infrastructure mapping - Threat actor attribution and TTPs - Attack chain reconstruction - Detection and defense mechanisms - Technical indicator processing You excel at: - Identifying meaningful relationships in security reports - Converting complex attack patterns into clear paths - Handling technical indicators safely (defanging, escaping) - Grouping related entities and actions - Maintaining technical precision while being concise You always: - Focus on specific, actionable intelligence - Preserve technical details - Structure output as valid JSON - Follow security best practices for indicator formatting - Avoid generic or filler language You understand cyber threat intelligence concepts, MITRE ATT&CK framework, and common attack patterns.
generate_knowledgebase	You are tasked with extracting relationship paths from security threat reports. Review the provided text and create paths that show how entities relate to each other. Each path should represent a meaningful chain of relationships. Rules: 1. Use format: A -> relationship -> B -> relationship -> C 2. Group related entities/relationships: A -> rel1, rel2 -> B, C 3. Use clear, concise relationship descriptions 4. Focus on significant relationships that tell a story 5. Include threat actors, malware, victims, tools, and techniques 6. Include different perspectives: geographical locations, motivations, tools, victims, and strategic context but keep focus on technicalities 7. Escape special characters in JSON (we need to parse the JSON with python) and be careful about special unicode characters. If needed, replace them 8. Defang URLs and IPs (replace . with [.] , @ with [@], http with hxxp, etc.) 9. Wrap output in ANSWER tags 10. Format output as valid JSON wrapped in ANSWER tags. Make sure to properly escape JSON characters. Remember this needs to be parsed as a JSON. Remove or replace hidden characters or anything that might break JSON parsing. Text: {{ few_shot_example }} Text: {{ input_text }} Output:
question_prompt.jinja	You are a cybersecurity expert creating advanced threat intelligence questions that require multi-hop reasoning across synthesized knowledge paths AND deep cybersecurity domain expertise. Context: Questions are based on synthesized knowledge paths extracted from threat intelligence reports. The paths serve as a "study guide" - analysts with access to these paths can more easily connect attack components, while those with only the full report would need to extract these relationships themselves. However, questions should be designed so they CANNOT be answered by paths alone OR domain expertise alone - both are required. Core Requirements: **Dual Dependency - Paths + Expertise**: Questions must require BOTH knowledge paths AND cybersecurity expertise: - Use specific path connections as technical foundation (provides the "what") - Require advanced cybersecurity expertise to interpret implications (provides the "so what") - Neither paths alone nor expertise alone should be sufficient to answer - Combine technical facts from paths with professional judgment from expertise **Path-Dependent Multi-Hop Reasoning**: Questions must explicitly use and connect multiple provided paths: - Follow path connections across 3+ hops to establish technical relationships - Reference specific artifacts, techniques, and infrastructure from paths - Connect distant path elements through logical progression - Use path details as foundation for expert analysis **Deep Domain Expertise Required**: Questions must require advanced cybersecurity knowledge to interpret path implications: - Understanding of malware analysis techniques and forensic methodologies - Knowledge of MITRE ATT&CK technique variations and detection challenges - Professional judgment about detection efficacy, operational priorities, and threat actor behaviors - Expertise in threat hunting methodologies and incident response strategies **Technical Sophistication**: Focus on expert-level analysis: - Professional decisions about defensive strategies based on path-revealed attack patterns - Forensic artifact correlation requiring both technical details and analytical expertise - Risk assessment combining path-based threat capabilities with expert threat landscape knowledge - Operational implications that require both technical facts and professional experience **Answer Format**: - Exactly 5 options (A-E) - Correct answer is a list: ["A"], ["A","C"...], ["C","D","E"...], etc. - Include sophisticated distractors requiring expertise to eliminate - Test understanding of WHY certain approaches are superior given the path context **Expertise Barrier**: - Questions should be difficult even WITH the paths provided - Require cybersecurity professionals to interpret what path connections mean operationally - Test judgment calls that combine technical path data with expert knowledge - Include nuanced decisions about priorities, methodologies, and implications Output: Valid JSON with proper escaping following the provided example format. Example knowledge base: {{ few_shot_example }} Knowledge base: {{ base }} Generate 6 challenging questions following this exact format. Output:
validate.jinja	You are validating and improving cybersecurity questions. Review the question against the knowledge base and either approve it or improve it. Requirements: 1. **Knowledge Base + Domain Expertise**: Questions must combine provided knowledge paths with advanced cybersecurity domain expertise - casual users cannot answer even with the paths 2. **Multi-Hop Reasoning with Both Sources**: Questions require connecting multiple knowledge paths AND applying domain expertise: - Follow 3+ knowledge path connections to establish technical facts - Apply cybersecurity expertise to fill gaps between path elements - Make professional inferences that combine path data with domain knowledge - Draw conclusions that require both path information AND expert interpretation 3. **Answer Format Validation**: - Exactly 5 options (A-E) - Correct answer must be a list: ["A"], ["A","C"], ["B","D","E"], etc. - "None" only as option E when contextually appropriate - Questions ask for multiple valid approaches, best combinations, most likely scenarios 4. **Expertise Barrier**: - Paths provide technical facts, but expertise is needed to interpret implications - Require advanced cybersecurity knowledge to connect path elements meaningfully - Test professional judgment on what connected paths mean operationally - Include sophisticated distractors requiring domain knowledge to eliminate 5. **Technical Rigor**: - Reference specific artifacts from paths (hashes, file paths, domains, IPs, techniques) - Use exact details from knowledge base - Create expert-level distractors with subtle technical flaws - Ensure reasoning requires both path following AND expert knowledge Validation Focus: - **Dual Dependency**: Questions must be difficult even WITH the paths - expertise still required - **Path Following**: Use specific path connections as foundation - **Expert Gaps**: Require domain knowledge to bridge between path elements - **Professional Judgment**: Test decisions that need both technical facts and expert interpretation - **Operational Implications**: Connect path relationships to security operations through expertise Multi-Hop Structure: - **Step 1**: Follow path connections to establish technical relationships - **Step 2**: Apply cybersecurity expertise to interpret what those relationships mean - **Step 3**: Make professional judgments about implications/actions based on combined analysis Common Issues to Fix: - Questions answerable by casual users even with paths provided - Pure path following without requiring expert interpretation - Missing professional judgment components that need domain expertise - Obvious answers that don't require cybersecurity knowledge to identify Improvement Guidelines: - **Path Foundation**: Start with specific path connections as technical foundation - **Expert Interpretation**: Require cybersecurity knowledge to understand implications - **Professional Decisions**: Test judgment calls combining paths with expertise - **Operational Context**: Connect technical path details to expert-level defensive actions If question cannot be validated against knowledge base, create a completely new one using the same format structure but based entirely on the provided paths requiring expert interpretation. If no changes are needed, keep the question exactly as-is. Output valid JSON with proper escaping. WRAP the answer in <ANSWER> tags. Example Output: {{ few_shot_example }} Knowledge Base: {{ knowledgebase }} Question to Validate: {{ question }}

Figure 22 Template names for relationship based generation.